

C Programming Book

Mohan Arunachalam

Contents

1	Introduction to C	2
1.1	List of all operators	3
1.2	Assignment operator	4
1.3	Arithmetic Operators	4
1.4	C-Tokens	6
1.5	printf function	6
1.6	Declaration Rules	8
1.7	Relational & Logical Operators	8

Chapter 1

Introduction to C

- Comparison between English language and C programming language

English Language	C Language
1. Alphabets (a, b, c, ...)	1. Keywords (32)
2. Words (More than 10K)	2. Operators (45)
3. Sentence (Grammer makes this)	3. Separators (14)
	4. Constants

- Constants

Item	value	C datatype
Integer	5 , -5 , 0	int
real	5.12 , 0.39 , 5.00 , -3.25	float
charater	'a' , '5'	char

- Reason for the name float but why not real?
 - 100. 231452156
 - * . is decimal point
 - * 231452156 : This keep on changing So float
- Primary / Primitive / Fundamental / Basic data types in C
 - Data is a meaningfull information
 - Datatype is a different kind of data like int , float and Char

Format Specifier	Data Type	Description
%c	char	Prints a single character
%s	char *	Prints a string
%d	int	Signed decimal integer
%i	int	Signed integer (same as %d in printf)
%u	unsigned int	Unsigned decimal integer
%hd	short int	Signed short integer
%hu	unsigned short int	Unsigned short integer
%ld	long int	Signed long integer
%lu	unsigned long int	Unsigned long integer
%lld	long long int	Signed long long integer
%llu	unsigned long long int	Unsigned long long integer
%f	float	Floating-point number (printf promotes float to double)
%lf	double	Double-precision floating-point (scanf uses %lf for double)
%Lf	long double	Long double floating-point

Format Specifier	Data Type	Description
%e / %E	float, double	Scientific notation
%g / %G	float, double	Shortest representation of %f or %e
%x	unsigned int	Hexadecimal integer (lowercase)
%X	unsigned int	Hexadecimal integer (uppercase)
%o	unsigned int	Octal integer
%p	void *	Pointer address
%%	—	Prints % character
%zu	size_t	Unsigned size type
%td	ptrdiff_t	Pointer difference type
%jd	intmax_t	Largest signed integer type
%ju	uintmax_t	Largest unsigned integer type

1.1 List of all operators

Operator	Description	Associativity
() [] . -> ++ --	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
* / %	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= > >=	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to or not equal to	left to right
&&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
 	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
 	Logical OR	left to right
? :	Ternary operator	right to left
= += -= *= /= %= &= ^= = <<= >>=	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	comma operator	left to right

Figure 1.1: Operators

1.2 Assignment operator

- It requires 2 arguments
- Left side arg must be variable
- Right side arg can be a variable / constant / expression
- Left side value is called l-value

```
1 // Not valid
2 a = 5
3 a = ;
4   = 10;
5 10 = 20;
6
7 // Valid
8 a = 5;
9 b = a;
10 c = 2 + 3;
```

- Every expression is replaced with constant

```
1 a = 5;
2 b = -3;
3 c = 9;
4
5 a = b; // a = -3
6 b = -c; // b = -9
7 a = -b; // a = 9
8
9 -c = a; // Error because l-value is expression not a variable
```

- In C language each and every statement ends with `;`
- Semicolon is called statement terminator (or) end of statement
- Why `.` is not used as terminator?
 - because `.` is used as decimal point in float number
- What is l-value error?
 - On the left hand side of assignment operation we should provide a variable but by mistake if we provide constant (or) expression then we get l-value required error.

1.3 Arithmetic Operators

```
1 // Example 1: Expression evaluation
2 a = 2 + 3;
3 a = 2 + 3 + 4;
4 a = 6 - 2 + 3 - 4 + 1;
5 a = 2 + 3 * 5;
6 a = (2 + 3) * 5;
7 a = 2 * 3 + 3 * 2;
```

Operation on	Result
int with int	int
int with float	float
float with int	float
float with float	float

```
1 // Example 2:
2 a = 5 / 2; // 2
3 a = -5 / 2; // -2
4 a = 5 / -2 // -2
```

```

5  a = -5 / -2 // -2
6
7  a = 5.0 / 2; // 2.5
8  a = 5 / 2.0; // 2.5
9  a = -5 / 2.0 // -2.5
10 a = 2 / 5 // 0
11 a = 2 / -5 // 0

```

```

1  // Example 3:
2
3  a = 17 / 3 / 2;
4  /*
5  a = 5 / 2
6  a = 2
7  */
8
9  a = 15 / 2 * 3;
10 /*
11 a = 7 * 3
12 a = 21
13 */
14
15 a = 9 * / 4;
16 /*
17 a = 27 / 4
18 a = 6
19 */
20
21 a = 6 * 3 / 4 * 5;
22 /*
23 a = 18 / 4 * 5
24 a = 4 * 5
25 a = 20
26 */

```

Division	Result of division	Modulus	Result of modulus
+ / +	+	+ % +	+
+ / -	-	+ % -	+
- / +	-	- % +	-
- / -	+	- % -	-

- Floating point numbers cannot used in modulus only integer is acceptable

```

1  // Example 1
2
3  a = 5 / -2; // -2
4  a = 5 % -2; // 1

```

```

1  // Example 2
2
3  a = 3.5 * 2 % 7; // Compilation error

```

- The result of modulus operation is remainder
- Result sign is same as numerator sign
 - If numerator is +ve then result is +ve
- Result sign does not depend on denominator
- Condition 1 : $x \% y == 0$ means then we can say that x is multiple of y
- Condition 2 : Numerator % 10 gives last digit of numerator

- **Condition 3** : When numerator is smaller then denominator then result is numerator
- **Condition 4** : When numerator is equal to denominator then result is zero

1.4 C-Tokens

- Smallest individual unit in C language is called C-Token
- C-Token are
 - Keywords
 - Operators
 - Separators
 - Constants
 - Identifiers
- There can be 'N' no. of space / Tabs / Newlines between two C-Tokens
- Program should be in good indentation

```
1 void main()
2 {
3     printf("Hello");
4 }
```

- The above program has following C-Tokens
 - Keyword: `void`
 - Operator: `(` `)` `(` `)`
 - Separator: `{` `;` `}`
 - Constant: `"Hello"`
 - Identifiers: `main` `printf`
- Total **11** C-Tokens are there in above program

1.5 printf function

- It will print 1st arg on screen
- 1st arg must be in pair of double quotes
- if more than one arg, then every arg must be separated by commas

```
1 printf("Hello");
2 // Hello
3
4 printf("    Hello    ");
5 // ....Hello....
6 // (Spaces are retained)
7
8 printf("Hello%dadc%d", 10, 20);
9 // Hello10abc20
10
11 printf("%d %d %d", 10, 20, 30);
12 // 10 20 30
13
14 printf("%d%d%d", 10, 20, 30);
15 // 102030
16
17 printf("%d,%d,%d", 10, 20, 30);
18 // 10,20,30
19
20 printf("%d %d %d", 10, 20);
21 // 10 20 <GV/Junk>
22
23 printf("%d %d %d", 10);
```

```

24 // 10 GV GV
25
26 printf("%d %d %d");
27 // GV GV GV
28
29 printf("%d %d %d", 10, 20, 30, 40);
30 // 10 20 30
31
32 printf("%d %f %c", 10, 3.75, 'a');
33 // 10 3.75 a
34
35 printf("%d", 5.5);
36 // GV
37
38 printf("%f", 5);
39 // GV
40
41 printf("%d", 5 + 2);
42 // 7
43
44 printf("5 + 2");
45 // 5 + 2
46
47 printf("%d + %d", 5 + 2);
48 // 7 + GV
49
50 printf("%d * %d = %d", 5, 2, 5+2);
51 // 5 * 2 = 7
52
53 printf("%d * %d = %d", 5, 2, 5*2);
54 // 5 * 2 = 10
55
56 printf("%f", 5/2);
57 // GV
58
59 printf("%d", 5/2);
60 // 2
61
62 printf("%f", 5.0 % 2);
63 // Error
64
65 printf("%d", -5 % -2);
66 // -1
67
68 printf("Hello", "Hai", "Bye");
69 // Hello
70
71 printf("Hello"Hai"Bye");
72 // HelloHaiBye
73
74 printf("Hello");
75 // Error
76
77 printf("Hello");
78 // Hello
79
80 printf("Hello");

```



```

81 //Error
82
83 printf("%d %d %d", "%d %d", 10, 20, 30, 40, 50);
84 // GV 10 20
85
86 printf("10, 20", "%d");
87 // 10, 20
88
89 int ts = 7500;
90 printf("ts");           // ts
91 printf(ts);             // Error
92 printf("Total sal = %d", ts); // Total sal = 7500
93 printf("%d + 1000", ts);  // 7500 + 1000
94 printf("%d", ts + 1000);  // 8500

```

1.6 Declaration Rules

```

1 void main()
2 {
3     int a; // Always say datatype to a variable
4     a = 5;
5     printf("%d", a);
6 }
7
8 // Output
9 // 5
10
11 void main()
12 {
13     int i;
14     float f;
15     i = 5;
16     f = 5.5;
17     printf("%d %f", i, f);
18 }
19
20 // Output
21 // 5 5.5

```

1.7 Relational & Logical Operators